Timers, Timing and PWM

Introduction to ARM Cortex-M Microprocessors — Chapter 10

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

Why This Matters

Why Software Delays Fail

Timer Architecture

Alternate Functions

PWM

More Code Examples

Summary

Design Challenge

Why This Chapter Matters

Every program so far has used `delay_ms()` — a loop that counts down and does nothing. It is wrong in three ways: its timing changes if you change the compiler optimization level or the clock frequency, it consumes 100% of the CPU while waiting, and it cannot be calibrated accurately. A hardware timer solves all three. It counts independently of the processor, generates precise intervals, measures elapsed time exactly, and produces the pulse-width modulated signals that dim LEDs and drive motors.

Objectives

After completing this chapter, you should be able to:

1. Explain how a prescaler and auto-reload register set a timer's period.
2. Calculate the prescaler and reload values needed for a required timing interval.
3. Configure a general-purpose timer for periodic update interrupts.
4. Use SysTick to maintain a millisecond system time base.
5. Select the correct alternate function number to route a pin to a timer channel.
6. Calculate PWM frequency and duty cycle, and configure a timer output compare channel to generate one.
7. Apply PWM to control LED brightness and motor speed.

Three Things Wrong With `delay_ms()`

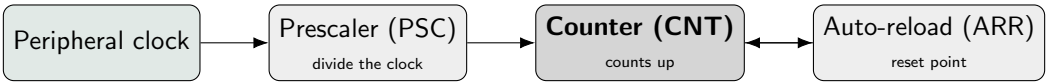
```
void delay_ms(int n) {
  for (volatile int i = 0;
       i < n * 4000; i++) ;
}
```

1. Its timing changes with the compiler optimisation level and the clock frequency
2. It consumes 100% of the CPU while waiting
3. It cannot be interrupted accurately — any ISR that fires makes the delay longer

A timer is a counter in hardware

It runs independently of the processor, at a rate you configure. It does not care what the CPU is doing, what the optimiser did, or how many interrupts fired. That is what makes it a clock rather than a guess.

Prescaler and Auto-Reload



$$f_{\text{event}} = \frac{f_{\text{CK}}}{(\text{PSC} + 1) \times (\text{ARR} + 1)}$$

The $+1$ is because a value of 0 means "divide by 1" — a counter that reloads at 0 still counts one tick.

A Timing Calculation

Required
A 1 Hz interrupt from a 16 MHz clock.

Choose a prescaler
PSC = 15999 divides 16 MHz down to 1 kHz — a convenient 1 ms tick.

Then the reload value
ARR = 999 gives 1000 ticks = 1 s.

$$\text{Check: } \frac{16{,}000{,}000}{16000 \times 1000} = 1\,\text{Hz}$$

```
TIM2->PSC = 15999;
TIM2->ARR = 999;
```

Why choose PSC first

Both registers are 16-bit, so neither can exceed 65535. Pick the prescaler to give a convenient tick — 1 ms or 1 µs — then the reload value is just the number of ticks you want.

A Periodic Timer Interrupt

```
RCC->APB1ENR |= (1U << 0);        // TIM2 clock
TIM2->PSC = 15999;                 // 16 MHz -> 1 kHz
TIM2->ARR = 999;                   // 1000 ticks -> 1 Hz
TIM2->DIER |= (1U << 0);           // update interrupt enable
TIM2->CR1  |= (1U << 0);           // start counting
NVIC_EnableIRQ(TIM2_IRQn);

void TIM2_IRQHandler(void)
{
    if (TIM2->SR & (1U << 0)) {    // update flag?
        TIM2->SR &= ~(1U << 0);    // clear it
        GPIOA->ODR ^= (1U << 5);
    }
}
```

The CPU is now free between interrupts — it can sleep, or do real work, and the LED still blinks at exactly 1 Hz.

SysTick — The System Time Base

- A 24-bit down-counter built into the **core**, not the vendor's peripherals.
- Present on **every** Cortex-M, at the same address — so code using it is portable.
- Normally configured for a 1 ms tick.

```
SysTick_Config(SystemCoreClock / 1000);
```

```
volatile uint32_t ticks = 0;

void SysTick_Handler(void)
{
    ticks++;
}

uint32_t millis(void)
{
    return ticks;
}
```

This is what replaces `delay_ms()`

Instead of *waiting*, record when something last happened and check whether enough time has passed. That is the non-blocking pattern Chapter 14 builds a whole system from.

Routing a Pin to a Peripheral

AF0	(system)
AF1	(TIM1/TIM2)
AF2	(TIM3..5)
AF3	(TIM8..11, CEC)
AF4	(I2C1..4, CEC)
AF5	(SPI1/2/3/4)
AF6	(SPI2/3/4, SAI1)
AF7	(SPI2/3, USART1..3, UART5, SPDIF-IN)
AF8	(SPI2/3, USART1..3, UART5, SPDIF-IN)
AF9	(CAN1/2, TIM12..14, QUADSPI)
AF10	(SAI2, QUADSPI, OTG_HS, OTG_FS)
AF11	
AF12	(FMC, SDIO, OTG_HS$^{(1)}$)
AF13	(DCMI)
AF14	
AF15	(EVENTOUT)

A pin can serve several peripherals. `MODER` set to `10` selects alternate-function mode; `AFR` then chooses which of up to sixteen functions.

Selecting the Function Number

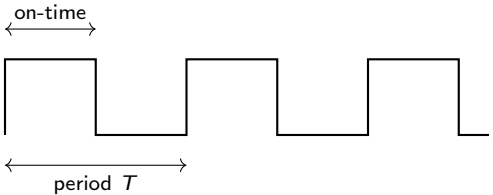
- Four bits per pin, so `AFR[0]` covers pins 0–7 and `AFR[1]` covers pins 8–15.
- For pin $n \geq 8$ the shift is $(n - 8) \times 4$.

```
// PA5 -> TIM2_CH1 is AF1
GPIOA->MODER &= ~(3U << 10);
GPIOA->MODER |=  (2U << 10);

GPIOA->AFR[0] |= (1U << (5*4));
```

The function numbers are in the **datasheet**, not the reference manual — a distinction that costs people an afternoon.

Pulse Width Modulation



$$\text{duty cycle} = \frac{\text{on-time}}{T} \qquad f_{\text{PWM}} = \frac{f_{\text{CK}}}{(\text{PSC}+1)(\text{ARR}+1)}$$

A digital pin producing an analog effect

The pin is only ever fully on or fully off — but switch it fast enough and an LED looks dimmed, a motor runs slower, a heater delivers less power. The average is what the load responds to.

Configuring PWM

```
TIM2->PSC  = 15;       // 16 MHz -> 1 MHz
TIM2->ARR  = 999;      // 1 MHz / 1000 = 1 kHz PWM
TIM2->CCR1 = 250;      // 250/1000 = 25% duty

TIM2->CCMR1 |= (6U << 4);  // PWM mode 1 on channel 1
TIM2->CCMR1 |= (1U << 3);  // preload enable
TIM2->CCER  |= (1U << 0);  // enable the output
TIM2->CR1   |= (1U << 0);  // start
```

ARR sets the frequency, CCR sets the duty

Change `CCR1` at any time and the duty cycle changes on the next cycle — no glitch, because the preload register updates only at the reload point. That is what makes smooth dimming possible.

A higher PWM frequency is less visible but wastes more switching energy. For LEDs, 1 kHz is comfortably above flicker perception.

The Bad `delay_ms()` — For Comparison

```c
void delay_ms_bad(int n)
{
    for (volatile int i = 0; i < n * 4000; i++)
        ;
    /* Wrong in three ways: drifts with optimisation level
       and clock frequency, burns 100% CPU, and any ISR that
       fires makes it longer than requested. Do not use this
       in real designs. */
}
```

A Proper `delay_ms()` — Built on SysTick

```c
static volatile uint32_t systick_ms = 0;

void SysTick_Handler(void) { systick_ms++; }

void systick_init(uint32_t core_clock_hz)
{
    SysTick_Config(core_clock_hz / 1000U); // 1 ms tick
}

uint32_t millis(void) { return systick_ms; }

// Blocking, but exact regardless of compiler optimisation:
// it counts real elapsed ticks, not a fixed loop count.
void delay_ms(uint32_t n)
{
    uint32_t start = millis();
    while ((millis() - start) < n) {
        __asm volatile ("wfi");  // sleep between ticks
    }
}
```

An Alternative: `delay_ms()` on a General-Purpose Timer

```c
void tim2_delay_init(void)
{
    RCC->APB1ENR |= (1U << 0);  // TIM2 clock enable
    TIM2->PSC = 15999;          // 16 MHz -> 1 kHz
    TIM2->ARR = 0xFFFF;         // free-running
    TIM2->CR1 |= (1U << 0);     // start counting
}

void delay_ms_tim2(uint32_t n)
{
    TIM2->CNT = 0;
    while (TIM2->CNT < n)
        ;
}
```

Useful when SysTick is already claimed by an RTOS, and the mechanism is the same prescaler/auto-reload arithmetic from earlier in this chapter.

The Non-Blocking Pattern (Preview of Chapter 14)

```c
typedef struct {
    uint32_t last_ms;
    uint32_t period_ms;
} periodic_task_t;

int task_due(periodic_task_t *t)
{
    if ((millis() - t->last_ms) >= t->period_ms) {
        t->last_ms += t->period_ms;  // schedule next, no drift
        return 1;
    }
    return 0;
}

// Example: blink an LED every 500 ms without ever blocking
void non_blocking_blink_demo(void)
{
    static periodic_task_t blink = { .last_ms = 0, .period_ms = 500 };
    for (;;) {
        if (task_due(&blink)) GPIOA->ODR ^= (1U << 5);
        /* ... other work happens here every loop iteration ... */
    }
}
```

PWM Duty-Cycle Helper

```c
void pwm_init_tim2_ch1(uint32_t psc, uint32_t arr)
{
    RCC->APB1ENR |= (1U << 0);
    TIM2->PSC = psc;
    TIM2->ARR = arr;
    TIM2->CCR1 = 0;                  // start at 0% duty
    TIM2->CCMR1 |= (6U << 4);        // PWM mode 1, channel 1
    TIM2->CCMR1 |= (1U << 3);        // preload enable
    TIM2->CCER  |= (1U << 0);        // enable output
    TIM2->CR1   |= (1U << 0);        // start
}

// percent: 0-100
void pwm_set_duty_percent(uint8_t percent)
{
    if (percent > 100) percent = 100;
    uint32_t arr = TIM2->ARR;
    TIM2->CCR1 = ((uint32_t)percent * (arr + 1)) / 100U;
}
```

Example: A Breathing LED, Non-Blocking

```
void led_breathe_demo(void)
{
    static periodic_task_t step = { .last_ms = 0, .period_ms = 20 };
    static int duty = 0;
    static int dir  = 1;

    if (task_due(&step)) {
        duty += dir;
        if (duty >= 100 || duty <= 0) dir = -dir;
        pwm_set_duty_percent((uint8_t)duty);
    }
}
```

Every 20 ms the duty cycle steps by one percentage point up or down, driven entirely by `task_due()` — no blocking call anywhere in the loop.

Design-Challenge Skeleton: One Time Base, Three Tasks

```
static periodic_task_t sensor_task  = { .last_ms=0, .period_ms=10   };
static periodic_task_t display_task = { .last_ms=0, .period_ms=100  };
static periodic_task_t log_task     = { .last_ms=0, .period_ms=1000};

void read_sensor(void)    { /* ... */ }
void update_display(void) { /* ... */ }
void write_log(void)      { /* ... */ }

void scheduler_loop(void)
{
    for (;;) {
        if (task_due(&sensor_task))  read_sensor();
        if (task_due(&display_task)) update_display();
        if (task_due(&log_task))     write_log();
    }
}
```

One SysTick-driven `millis()` schedules the sensor (10 ms), display (100 ms) and log (1 s) tasks from a single interrupt source, without drift.

Summary

- A software delay is wrong three ways: it depends on the **optimiser** and **clock**, it burns the **whole CPU**, and interrupts **stretch** it.
- A timer is a **hardware counter**: $f = f_{CK}/((PSC+1)(ARR+1))$.
- Choose **PSC** for a convenient tick, then **ARR** is the number of ticks.
- **SysTick** is in the core, identical on every Cortex-M, and is what a portable `millis()` is built on.
- An **alternate function** needs `MODER = 10` *and* the right number in `AFR` — four bits per pin.
- **PWM**: `ARR` sets the frequency, `CCR` sets the duty cycle. A digital pin produces an analog effect.
- The **non-blocking pattern** (`millis()` + `task_due()`) replaces `delay_ms()` in real designs and scales to any number of periodic tasks.

Next: Chapter 11 — ADC and Analog Interfacing

You can produce an analog effect. Next: measuring a real one.

Design Challenge (To be worked on paper, before any code)

The brief

A system must perform three periodic tasks: read a sensor every 10 ms, update a display every 100 ms, and write a log entry every 1 s.

Design the timing structure using a single hardware time base. Specify the timer's PSC and ARR values for a 16 MHz clock, and show how one interrupt can schedule all three tasks without drift. Then determine what happens if the log write takes 40 ms, identify which requirement breaks first, and give two different ways to fix it — one that changes the schedule and one that changes the architecture.

Extension — a listening device. Now add a fourth activity to the same processor: sampling a microphone at 16 kHz for keyword spotting, as in audio timing. On paper, determine whether the system above can still meet all its deadlines. Produce a cycle budget at 16 MHz showing what fraction of the processor the sampling ISR alone consumes; decide which of the four activities may be delayed and which may not, and justify the ranking in terms of what breaks when each one slips. State where the audio buffer lives and how the main loop knows a window is ready without racing the ISR. Finally, identify the one activity in the original three that is now unsafe to leave as written, and say what you would change about it — without adding a second processor.

Questions?

Introduction to ARM Cortex-M Microprocessors — Chapter 10